PROJECT DOCUMENTATION

Version 3.3

Work Log

Daily Work Report Tracker

A daily task & attendance logger with automated weekly reporting

Offline edition · Cloud edition (hosted, multi-user)

Prepared by	**Manish Kumar Sabbani — Technical Support Analyst**
Organisation	**TechSquad LLC**
Version	**3.3 (per-task end user & opt-in login persistence)**
Date	**11 September 2026**
Status	**Complete, hosted & in daily use**
Live at	**https://manishsabbani2606.github.io/Task-Tracker-/**
Deliverable	**Offline: task-tracker-2.0.html · Cloud (hosted): worklog-cloud.html**

1 Executive Summary

The Daily Work Report Tracker (“Work Log”) is a lightweight web application that lets me log my daily support tasks and working hours, and export a formatted weekly report for management review.

It was built to replace ad-hoc note-taking with a consistent, structured record of daily activity. Each day I clock in and out, log the tasks I worked on — with ticket numbers, category, SLA priority, status, and the **end user** I helped — and the tool keeps a running tally of hours and task time. At the end of each week, a single click produces a professional PDF report summarising attendance and every task, branded for TechSquad LLC.

The entire application is a single HTML file that runs in any modern web browser. It requires no installation and no server. As of version 2.1 it also supports individual user accounts, so the same file can be shared with others — each person signs in to a private profile and sees only their own timesheets.

Version 3.0 adds a cloud edition (**worklog-cloud.html**): the same tool, hosted online for free, with real sign-in and data saved to a central database so every user's timesheet is kept safely and is available from any device — with an admin view of the whole team (see §9). Version 3.3 adds a per-task **end-user** field (the person I assisted) and makes the cloud sign-in **opt-in** — the session no longer persists after the browser is closed unless “Keep me signed in” is ticked (see §6).

What it does	Daily task logging, clock in/out attendance, weekly PDF reporting
Who it is for	Any individual tracking day-to-day work for weekly updates — the owner and any external users the file is shared with
Accounts	Each user creates a profile (name, company, role, password) and signs in; profiles are kept separate
End user	Each task records the end user assisted, so the weekly report shows who each piece of work was for
How it runs	Open the HTML file in a browser (offline edition), or hosted online (cloud edition)
Privacy	Offline edition keeps everything on-device; the cloud edition stores data in a dedicated Firebase project

2 Purpose & Objectives

The project set out to deliver a simple, reliable tool that satisfies a recurring need: submitting an accurate weekly activity report to management.

- **Capture work consistently** — record each task with enough structure (ticket, category, SLA priority, status, end user, time) to be meaningful in a report.
- **Track attendance** — log clock-in and clock-out times and automatically calculate hours worked.
- **Automate weekly reporting** — turn a week of daily entries into a polished, presentation-ready PDF in one click.
- **Be effortless to use daily** — fast entry, no friction, and reliable day-to-day persistence.
- **Keep data private and safe** — on-device storage with a dependable backup path, or central cloud storage.

3 Key Features

Feature	Description
User accounts	Sign-up / sign-in with name, company, role and password; each user has a private profile.
Multiple profiles	Several people can use the same shared file without seeing each other's data; a Sign out button switches profiles.
Daily task log	Record each task with a description, optional ticket number, category, SLA priority, end user and time spent (minutes).
End user field new 3.3	Each task records the **end user** — the person assisted — so the report shows who the work was for. The report header still names me as the author.
Task table	The daily log shows tasks in a table — Task, User, Category, SLA Priority, Status, Time — that scrolls within its box on small screens.
Categories	Incident, Service Request, Support Ticket, Work, Meeting, Project, Learning, Admin, Other.
SLA Priority	Low, Medium, High, Urgent — colour-coded for quick scanning.
Status	Each task is Open, In Progress or Resolved, updated as work progresses.
Clock in / out	One-click clock-in and clock-out; attendance hours are calculated automatically.
KPI dashboard	Live cards for hours today, task time today, open tasks, and total hours this week.
Weekly hours chart	A bar chart of hours worked per day across the selected week.
Weekly PDF report	One-click export of a branded report: attendance summary plus a Task Log table (Date, Task, User, Category, SLA Priority, Status, Time).
Opt-in login new 3.3	**Keep me signed in** checkbox on the cloud sign-in screen; when unticked the session ends as soon as the browser or tab is closed (see §6).
Dark mode	Light/dark theme toggle that remembers the chosen preference.
Cloud sync cloud edition	Hosted online with real accounts; data saved centrally and available on any device.
Admin overview cloud edition	The admin can view and export every user's timesheet from one place.

4 User Guide — Daily Workflow

The intended daily routine takes under a minute of overhead:

1. **Open & sign in.** Open the tracker in the browser, then sign in to your profile (or create one on first use).
2. **Clock in.** On arrival, click *In* under Clock in (or type the time). Click *Out* when leaving.
3. **Log tasks through the day.** Type what was worked on, add a ticket number, pick a category and SLA priority, enter the **end user** assisted and the minutes, then click *Add task*. Tasks appear in the daily table.
4. **Update status.** As work moves along, switch a task between Open, In Progress and Resolved.
5. **Submit the day.** Click *Submit* to lock the day's attendance once finished.

The day navigator (‹ Today ›) moves between dates, so a missed entry can be back-filled at any time. All totals — attendance hours, task time, and the KPI cards — update automatically.

Producing the weekly report

1. In the Weekly report section, set *Week of* to the Monday of the week.
2. Click *Preview* to review the week on screen, or go straight to export.
3. Click *Export PDF*. The report is generated and saved to the Downloads folder, ready to send to the manager.

5 The Weekly Report

The exported PDF is designed to be handed directly to management. It contains:

- **Branded header** — “Weekly Work Report”, the week's date range, and the TechSquad LLC / name / role block. The header names me as the author; the *User* column names the end user for each task.
- **Summary cards** — total hours worked and total task time logged for the week.
- **Attendance Summary table** — clock-in, clock-out, attendance hours and task time for each day.
- **Task Log table** — every task for the week in a table with Date, Task, User, Category, SLA Priority, Status and Time columns; long descriptions wrap and the table carries its header across page breaks.
- **Page numbering** across multi-page reports.

> **SAMPLE PROVIDED**
> A sample weekly report PDF (**Weekly_Report_sample.pdf**) is included alongside this document to illustrate the exact output format.

6 User Accounts & Login

Version 2.1 added a sign-in layer so the tracker can be shared with multiple people — each keeps a private profile and sees only their own timesheets.

On first open, the user is presented with a **Create account / Sign in** screen. Creating an account captures the user's name, company, job title, email and a password; these details personalise the app and appear on that user's exported report. Signing in restores their saved history.

- **Separate profiles** — several people can use the same file, even on one browser, without seeing each other's data.
- **Personalised reports** — the signed-in user's name, company and role are written into their weekly PDF automatically.
- **Hashed passwords** — the offline edition stores passwords as a salted SHA-256 hash via the browser's Web Crypto API, never in plain text; the cloud edition delegates passwords to Firebase Authentication.

Session persistence & “Keep me signed in” new 3.3

Earlier versions kept a user signed in indefinitely — reopening or refreshing the page landed straight back in the app. From version 3.3 the sign-in is **opt-in**: by default a session lasts only as long as the browser is open, and a *Keep me signed in* checkbox lets the user choose to stay signed in on a trusted device.

Choice at sign-in	What happens
“Keep me signed in” unticked (default)	The session is remembered only until the browser (or the last tab of the site) is closed. A page *refresh* stays signed in, but fully quitting the browser signs the user out — they must sign in again next time.
“Keep me signed in” ticked	The session persists across browser restarts on that device until the user explicitly clicks *Sign out*. Intended only for a personal, trusted computer.

How it works

- **Cloud edition.** Firebase Authentication supports three persistence levels. Before signing in, the app calls `auth.setPersistence(...)` with `LOCAL` when “Keep me signed in” is ticked, or `SESSION` when it is not. `SESSION` keeps the token in the tab's *session storage*, which the browser discards when the browser is closed — so the login vanishes exactly as required. New sign-ups always use `SESSION`, so a freshly created account is never silently kept signed in.
- **Offline edition.** The same choice maps to browser storage: with “Keep me signed in” the session id is written to `localStorage` (survives a restart); without it, to `sessionStorage` (cleared when the browser closes). On start-up the app reads `sessionStorage` first, then `localStorage`, and signing out clears both.

EXISTING CLOUD SESSIONS

A session created before this change was stored with the old “always remember” behaviour and will stay signed in until the user clicks *Sign out* once. From the next sign-in onward the new opt-in behaviour applies.

IMPORTANT — APPLIES TO THE OFFLINE EDITION

In the offline single-file edition, accounts and their data live entirely in each user's own browser on their own device. This is a convenience login for keeping profiles separate — not server-grade authentication. Data is not

> transmitted to the file's author and does not sync across devices; each user should keep a backup file (see §8). The cloud edition (§9) removes these limitations with real server-side accounts and central storage.

7 Technical Overview

The application favours simplicity and portability over complexity. It is deliberately built as one file that will keep working for years with no maintenance or dependencies to manage.

Architecture	Single self-contained HTML file (markup, styles and logic in one document)
Languages	HTML, CSS, and vanilla JavaScript — no frameworks
PDF generation	jsPDF (client-side); the report is produced entirely in the browser
Charts & icons	Inline SVG — no image assets or external libraries
Accounts / passwords	Offline: local profiles in localStorage, passwords salted and hashed with SHA-256 via the Web Crypto API. Cloud: Firebase Authentication.
Session persistence	Cloud: Firebase `setPersistence` (LOCAL vs SESSION). Offline: localStorage vs sessionStorage — chosen by the “Keep me signed in” checkbox (§6).
Data storage	Offline: browser localStorage, namespaced per account, plus optional file storage (§8). Cloud: Cloud Firestore.
Server / hosting	Offline edition needs none; cloud edition served free on GitHub Pages
Connectivity	Offline edition works fully offline
Browser support	Any modern browser (Chrome, Edge, Safari, Firefox), desktop and mobile
Cloud edition	Adds Firebase Authentication + Cloud Firestore, hosted free on GitHub Pages (see §9)

Security & privacy

In the offline edition all data — including user profiles — stays on the local machine; the login is an on-device convenience gate (passwords are hashed, not stored in plain text) and is not a substitute for server-based authentication. The cloud edition uses Firebase Authentication and Firestore security rules for genuine, server-enforced access control.

8 Data Persistence & Backup

Reliable persistence was a core requirement: work logged today must be present tomorrow. In the offline edition this is handled at three levels.

1. Automatic in-browser save

Every action — clocking in, adding a task, changing a status — is saved immediately to the browser's local storage, with an on-screen “✓ Saved” confirmation. Reopening the same file in the same browser restores the full history.

2. Backup & restore file

The *Save backup file* button exports all data to a single `.json` file that can be kept in OneDrive. *Restore from file* loads it back — on the same computer or a different one — making the data portable and recoverable.

3. Optional auto-save to a file

On browsers that support it (Chrome and Edge), the tracker can be linked once to a data file; from then on every change is written straight to that file. Combined with OneDrive sync, this provides always-current, cross-device history with no manual steps.

> **RECOMMENDED SETUP**
> For the offline edition, open the tracker in Chrome or Edge and enable auto-save to a file inside the OneDrive project folder. For team use with central storage, use the cloud edition (§9).

9 Cloud Edition (Hosted & Multi-User)

Alongside the offline single-file tracker, a cloud edition (**worklog-cloud.html**) turns Work Log into a hosted, multi-user application. It keeps the identical interface, but replaces on-device storage with a real backend — so accounts are genuine, and every user's data is saved centrally and available on any device.

Authentication	Firebase Authentication (email & password) — real, server-side sign-in; passwords are handled and hashed by Google and never seen by the app
Session control	“Keep me signed in” selects Firebase LOCAL vs SESSION persistence, so a session ends when the browser closes unless the user opts in (§6)
Database	Google Cloud Firestore — each user's profile and daily logs are stored in the cloud
Privacy model	Security rules enforce that every user reads and writes only their own data, while the designated admin can read all users' timesheets
Admin overview	The admin account gets a “team timesheets” panel to open and export any user's weekly report (read-only)
Real-time sync	Live Firestore listeners — a user's data updates instantly across their devices, and the admin view updates live as users log work (no refresh)
Cross-device	Sign in from any device or browser and your data follows you automatically
Access control	Invite-only: only email addresses the admin adds to the invite list can create an account (enforced by the security rules). The admin manages the list from the app.
Hosting	Served as a static site on GitHub Pages (free), live at https://manishsabbani2606.github.io/Task-Tracker-/ — shared with users as a single link
Cost	Free — 10 users sits comfortably within the Firebase and GitHub Pages free tiers

Which edition to use

The offline single-file edition (§6) suits a personal log, or a quick share where each person keeps their own local copy. The cloud edition is the choice when data must be saved centrally, reached across devices, or reviewed by an admin across a team.

> **A NOTE ON THE FIREBASE CONFIGURATION**
>
> The cloud edition embeds a Firebase “web config” (an API key and project identifiers). This is public by design and safe to include in the page — access is enforced by the Firestore security rules, not by keeping the key secret.

10 Design & User Experience

Version 2.0 introduced a professional, enterprise-grade interface designed to look and feel like a real internal tool rather than a simple form.

- **Application shell** — a sticky top bar with a Work Log logo mark and the user's name, role and organisation.
- **KPI dashboard** — four summary cards giving an at-a-glance view of the day and week.
- **Data visualisation** — a clean weekly-hours bar chart with value labels and hover detail.
- **Responsive tables** — the daily task table scrolls within its own frame on narrow screens, so the page never overflows sideways; layout works on desktop and mobile.
- **Refined visual system** — a restrained slate-and-indigo palette, consistent SVG iconography, and clear typography.
- **Light & dark themes** — a fully designed dark mode, not an automatic inversion.

11 Version History

Version	Date	Summary of changes
1.0	Baseline	Core tracker: daily task log, clock in/out with hour calculation, weekly PDF export, and automatic local storage.
1.5	2 Sep 2026	Added ticket number, priority and status fields; expanded the category list; introduced a dark-mode toggle; added TechSquad LLC branding to the report.
1.8	2 Sep 2026	Persistence hardening: a visible “Saved” confirmation, backup & restore to a file, and optional auto-save directly to a file.
2.0	3 Sep 2026	Professional redesign: application top bar, KPI dashboard, weekly-hours chart, SVG iconography, a refined colour system, and matching report branding.
2.1	3 Sep 2026	User accounts & login: per-user local profiles, private data per profile, personalised reports, and sign out.
3.0	4 Sep 2026	Cloud edition (worklog-cloud.html): real Firebase Authentication, per-user data saved to Cloud Firestore, an admin view of all users, cross-device sign-in, and free GitHub Pages hosting.
3.1	4 Sep 2026	Daily tasks shown as a table; the weekly PDF gains a matching Task Log table; “Priority” renamed to SLA Priority; a responsive fix so the layout no longer overflows on mobile; branding standardised to “Work Log”. Both editions.
3.2	4 Sep 2026	Cloud edition went live on GitHub Pages; added real-time sync (live Firestore listeners); made the site invite-only with an admin-managed invite list enforced by the security rules; added a Forgot-password option.
3.3	11 Sep 2026	Per-task **end-user** field so each task records the person assisted (shown in the report's User column); **opt-in login persistence** — a “Keep me signed in” checkbox, with the session otherwise ending when the browser closes. Both editions; further CSS polish.

12 Limitations & Future Enhancements

The tool meets its objectives; the following are known boundaries and candidate improvements.

Area	Note / opportunity
Browser-bound storage	In the offline edition, in-browser data is tied to one browser on one computer. Mitigated by the backup file and optional auto-save (§8).
Auto-save browser support	Silent auto-save-to-file works in Chrome and Edge; Safari users use the manual backup button.
Offline vs cloud edition	The limitations above describe the offline single-file edition. The cloud edition (§9) resolves them — real accounts, central storage, cross-device access and an admin overview.
Cloud edition notes	A single admin is set by email in the security rules. The Firebase / Pages free tiers cover a small team comfortably; a Supabase alternative is also included (supabase-setup.sql).
Possible additions	Monthly reporting, search/filter across history, per-category time analytics, and a one-click browser launcher.

13 Deliverables

File	Description
worklog-cloud.html	Cloud edition — hosted, multi-user app with Firebase login and Firestore storage (v3.3).
firestore.rules	Cloud security rules (each user private; admin can read all) for the Firebase edition.
supabase-setup.sql	Alternative backend script (Supabase/Postgres) for the same hosted model.
task-tracker-2.0.html	Offline edition — the complete single-file application (v3.3, includes local login).
task-tracker.html	Working copy (kept in sync with the current version).
Weekly_Report_sample.pdf	A sample of the exported weekly report (with the Task Log table).
This document	Project documentation (PDF).